

Final Project Report

Course: CSCI 435 - Blockchain & Cryptosystems **Project Title:** CSCI435 HD Web Wallet (Browser Extension)

Author: Adi Sakhantayev (202137654) **Submission Date:** 25/04/2026

1. Introduction

This project develops a browser-extension Ethereum wallet prototype that supports onboarding, account management, and Sepolia transaction signing. A key problem in blockchain usability is balancing security with a smooth user experience, especially for first-time users who need to create/import wallets and safely sign transactions.

The importance of this problem comes from real security risks: seed phrases and private keys must be protected, while wallet operations should remain simple and understandable. This project addresses this by combining encrypted local storage, account controls, and blockchain RPC integration in one extension.

The project aims to provide a practical end-to-end wallet flow for coursework:

- Create or import wallet
- Unlock and manage accounts
- Check balance and send transactions
- Expose `window.ethereum` so dApps can make wallet RPC requests

2. Project Objectives

Main objectives:

- Implement wallet onboarding with both creation and import paths.
- Encrypt sensitive wallet data before local storage.
- Support unlock/lock workflow and active account selection.
- Enable Sepolia balance lookup and ETH transaction broadcast.
- Provide basic dApp compatibility via provider injection.

Expected functionality:

- User can create wallet from mnemonic or import via seed/private key.
- User can unlock wallet with password and access dashboard features.
- User can derive/import accounts and send transaction from chosen account.
- dApps can call supported RPC methods through injected provider.

Target users:

- CSCI 435 project author and evaluator.
- Beginner blockchain users/developers needing a wallet prototype.

3. System Overview

The system follows a modular browser extension architecture:

- **Popup UI (`src/popup/`):** onboarding, unlock, dashboard, send transaction flow, account controls.

- **Background service worker (`src/background/index.ts`):** RPC request handling, signer unlock logic, signing/sending operations.
- **Content script (`src/content/index.ts`):** bridge between webpage context and extension runtime.
- **Injected provider (`src/injected/provider.ts`):** exposes `window.ethereum` request API and events.
- **Utility modules (`src/utls/`):** mnemonic/account derivation, RPC helpers, encryption/decryption, storage persistence.

Technologies used: - TypeScript + React + Vite - **ethers** v6 for cryptography, wallet logic, and RPC operations - Chrome Extension APIs (`chrome.storage`, `chrome.runtime`, `chrome.notifications`) - Bun for development scripts

User interaction summary: 1. User sets up wallet in popup. 2. Secrets are encrypted and saved locally. 3. User unlocks wallet and selects active account. 4. User sends transaction or dApp sends RPC request through provider bridge.

3.1 Architecture Diagram

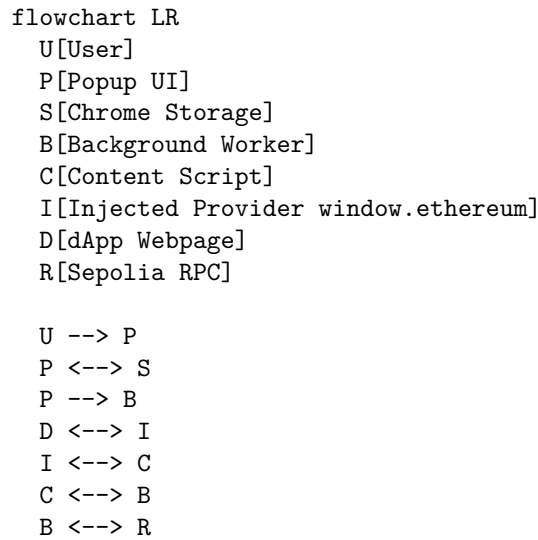


Figure 1: System Architecture

4. Project Workflow

Development process was milestone-driven:

- **Planning phase**
 - Defined core MVP: onboarding, secure storage, account management, send flow.
 - Planned and implemented workstreams across UI, provider bridge, security/storage, and docs/QA.

- **Milestone 1 (Working Prototype)**
 - Wallet create/import flows.
 - Unlock and local encrypted storage.
 - Balance retrieval and basic transaction path.
- **Milestone 2 (Feature Complete)**
 - `window.ethereum` provider integration.
 - RPC handling in background worker.
 - Improved validation, multi-account support, and docs updates.
- **Quality and debugging**
 - Iterative debugging of extension flows during implementation.

5. Smart Contract Design

This project focuses on wallet and transaction infrastructure rather than deploying a custom Solidity smart contract. Blockchain logic is implemented through transaction signing, RPC handling, and provider interactions.

5.1 Contract Structure

Equivalent design elements in this wallet system: - **State variables (off-chain app state):** active account, account source type, unlocked session status. - **Mappings/records:** encrypted wallet payload and imported account metadata in local storage. - **Event-style behavior:** provider events (`accountsChanged`, `chainChanged`) and runtime message callbacks. - **Access control pattern:** signing requires valid unlock session and user approval for sensitive requests.

5.2 Example Smart Contract Logic

Key blockchain-related functions in implementation: - `eth_accounts` / `eth_requestAccounts`: returns active address from unlocked signer. - `eth_getBalance`: fetches balance using Sepolia RPC provider. - `personal_sign`: prompts approval, signs message, returns signature. - `eth_sendTransaction`: prompts approval, signs/sends tx, returns tx hash. - `wallet_switchEthereumChain` / `wallet_addEthereumChain`: controlled behavior with Sepolia-only support.

6. Implementation Details

Programming languages: - TypeScript (extension logic and runtime modules) - TSX with React (popup UI)

Libraries and frameworks: - React 19 - Vite + CRXJS plugin - **ethers** v6 - Web Crypto API (PBKDF2 key derivation + AES-GCM encryption)

Implementation highlights: - Seed/private key encryption before storage. - Session password stored temporarily in `chrome.storage.session`. - Active ac-

count stored in session to coordinate popup and background signer selection. - Background worker rejects signing requests when wallet is locked.

7. Project Results

Current project results demonstrate that core goals are achieved: - Wallet onboarding works for create/import paths. - Encrypted storage and unlock flow are functional. - HD and imported account handling both operate in the dashboard. - Sepolia transactions can be signed and broadcast. - Basic dApp provider RPC handling works through extension bridge.

Evidence to include in final submission: - Screenshot set (onboarding, unlock, dashboard, send transaction, approval notification). - Example transaction hash and explorer link from Sepolia.

8. Individual Contribution Summary

Contributor	Responsibilities	Key Commits	Evidence Artifacts
Adi Sakantayev	End-to-end implementation: wallet onboarding, provider bridge/background RPC, encryption/storage security, and documentation/report assets	[Add hashes]	UI screenshots, dApp demo snippets, docs/presentation files

9. Challenges and Solutions

1. Challenge: secure local secret management

- Problem: plaintext seed/private keys are unsafe.
- Solution: implemented PBKDF2 + AES-GCM encryption and validated stored wallet shape before use.

2. Challenge: page-to-extension communication for dApp RPC

- Problem: webpage context cannot directly access extension internals.
- Solution: built injected provider + content script bridge + background RPC pipeline.

3. Challenge: mixed account source handling

- Problem: HD-derived and private-key-imported accounts require different signer restoration logic.
 - Solution: tracked account source metadata and restored appropriate signer dynamically.
4. **Challenge: transaction/signing safety**
- Problem: accidental or malicious requests can trigger signing without user awareness.
 - Solution: added explicit notification-based approval with reject/timeout behavior.

10. Limitations

- Network support is currently limited to Sepolia.
- RPC method support is intentionally limited to core wallet operations.
- Confirmation UI is basic (notification flow) and can be expanded with richer transaction details.
- No custom deployed Solidity contract is included in this version.
- Some modules (e.g., popup app state) can be further refactored for maintainability.

11. Future Work (Optional)

- Add multi-network support (testnets and configurable RPC).
- Expand provider compatibility and EIP support.
- Build detailed confirmation modal with gas/nonce/data decoding.
- Add stronger key-management options (hardware wallet support or secure enclave integration).

12. Conclusion

I successfully built a working browser-extension web wallet that integrates wallet UX, security-oriented local secret handling, and blockchain transaction functionality. The system demonstrates practical use of blockchain tooling and extension architecture concepts covered in CSCI 435.

The final prototype achieves core functional objectives and provides a clear foundation for production-level improvements in security, interoperability, and user experience.

13. References

1. Ethereum JSON-RPC API Documentation: <https://ethereum.org/en/developers/docs/apis/json-rpc/>
2. Ethers.js v6 Documentation: <https://docs.ethers.org/v6/>

3. BIP-39 Specification: <https://github.com/bitcoin/bips/blob/master/bip-0039.mediawiki>
4. Chrome Extensions Documentation: <https://developer.chrome.com/docs/extensions>
5. OWASP Cryptographic Storage Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html